COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 - Data Structures & Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	09/03/26	**DATE SUBMITTED:**	09/03/26
NAME:	Acala Arididdon Astudillo Antonio Bandigan	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO	**Seatwork 7.1 Using Sorting Algorithm**

1. PROCEDURE OUTPUT

1. What is sorting algorithm?

- A sorting algorithm is a method used to organize and rearrange elements in a list or an array into a specific order, such as ascending or descending. Sorting algorithms are essential in Data Structures and Algorithms because they improve data organization, which can be utilized by search algorithms such as Binary and Linear search operations in order to efficiently solve and manage large and complex datasets.

2.Where can sorting algorithms be used?

- **Databases & Spreadsheets:** Organizing records alphabetically or by date.
- **E-commerce:** Filtering products (e.g., sorting by "Price: Low to High").
- **Search Engines:** Ranking search results by relevance.
- **Operating Systems:** Scheduling tasks or managing files based on size or creation date.
- **Optimizing other algorithms:** Preparing data so algorithms like Binary Search can function correctly.

3. Explain the different types of sorting algorithms.

Bubble Sort - Compares neighboring elements and swaps them if they are in the wrong order. It repeatedly moves larger values towards the end.

Selection Sort - Finds the smallest element and places it in the correct position. It builds the final sorted array one item at a time.

Insertion Sort - Takes one element at a time and inserts it into its correct position. The algorithm virtually splits the array into two parts: a sorted region (on the left) and an unsorted region (on the right).

Quick Sort - Divides the list into smaller parts, sorts them, then combines them.

4. Give sample programs in C++ that use sorting algorithms, specifically selection sort, insertion sort, and bubble sort. Explain how the programs work.

Selection Sort

C/C++

```
https://www.scaler.com/topics/cpp-selection-sort/

#include <bits/stdc++.h>

using namespace std;


void selectionSort(int arr[], int n) {

  for (int i = 0; i < n - 1; i++) {

    int indexMin = i; // Taken first index as minimum.

    for (int j = i + 1; j < n; j++) {

      //Making comparisons

      if (arr[j] < arr[indexMin]) {

        // Updating the minimum value.

        indexMin = j;

      }

    }

    // Swapping the values using swap() method.

    swap(arr[indexMin], arr[i]);

  }

  cout << "Array after sorting:" << endl;

  for (int i = 0; i <= n - 1; i++) {
```

```
    // Printing the array after being sorted.
    cout << arr[i] << " ";
  }
}
int main() {
  int array[] = {10, 8, 5, 6, 9, 15, 2};
  int size = 7;
  selectionSort(array, size);
}
```

Explanation

The program implements selection sort using a nested for loop that goes through the whole array. It starts at the first element of the array and compares it to the rest to find the smallest element. Once the smallest element is found it then swaps that element with the first element. After that the nested for loop now starts at the second element and does the same process again, leading to a sorted array.

Best-Case Time Complexity: $O(n^2)$

Worst-Case Time Complexity: $O(n^2)$

Bubble Sort

C/C++

```
https://www.geeksforgeeks.org/cpp/bubble-sort-in-cpp/
// C++ program for the implementation of Bubble sort
#include <bits/stdc++.h>
using namespace std;
```

```
void bubbleSort(vector<int>& v) {
    int n = v.size();
    // Outer loop that corresponds to the number of
    // elements to be sorted
    for (int i = 0; i < n - 1; i++) {
        // Last i elements are already
        // in place
        for (int j = 0; j < n - i - 1; j++) {
                // Comparing adjacent elements
            if (v[j] > v[j + 1])
                // Swapping if in the wrong order
                 swap(v[j], v[j + 1]);
        }
    }
}
int main() {
    vector<int> v = {5, 1, 4, 2, 8};
    // Sorting the vector v
    bubbleSort(v);
    for (auto i : v)
```

```
        cout << i << " ";
    return 0;
}
```

Explanation

The program implements bubble sort using a nested for loop. The outer for loop handles how many times the vector is checked, while the inner loop compares the adjacent elements. It starts off by comparing the first element to its adjacent element and swaps if it has a greater value. This goes through the whole vector until, after the first pass, the largest element is at the end of the vector. After each pass the largest element moves to the end, leading to a sorted array.

Best-Case Time Complexity: $O(n^2)$

Worst-Case Time Complexity: $O(n^2)$

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the

permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.